

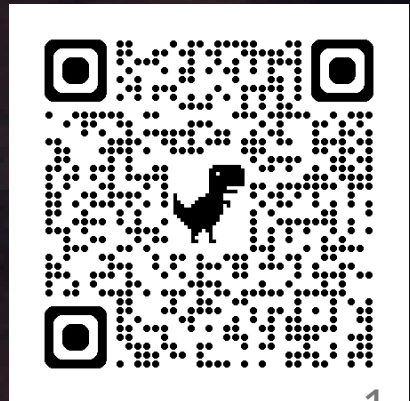
Every Map an Evolution, Every Room a Generation: Co-Evolution in a Procedurally-Generated Video Game

Erik Fredericks, frederer@gvsu.edu

GPTP, June 4th, 2026

<https://efredericks.github.io>

(Background gif: Binding of Isaac)



Premise

Create a fully-evolved two-dimensional rogue/lite/ game

All aspects of procedural content generation (PCG) managed by genetic programming (GP)

- Map creation
- Entity placement
- Enemy AI control

s. "I am in your debt. I will grant one wish



inator St:18/07 Dx:17 Co:18 In:15 Wi:11 Ch:17
32(132) Pw:56(56) AC:-4 Xp:13/51018 T:44408 Bu

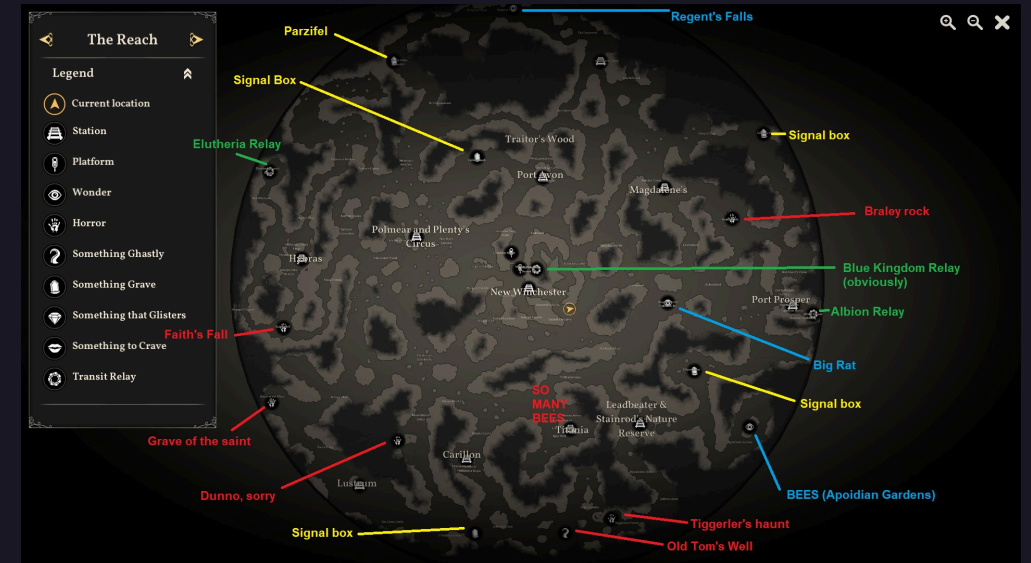
Status

One GP active and one in progress

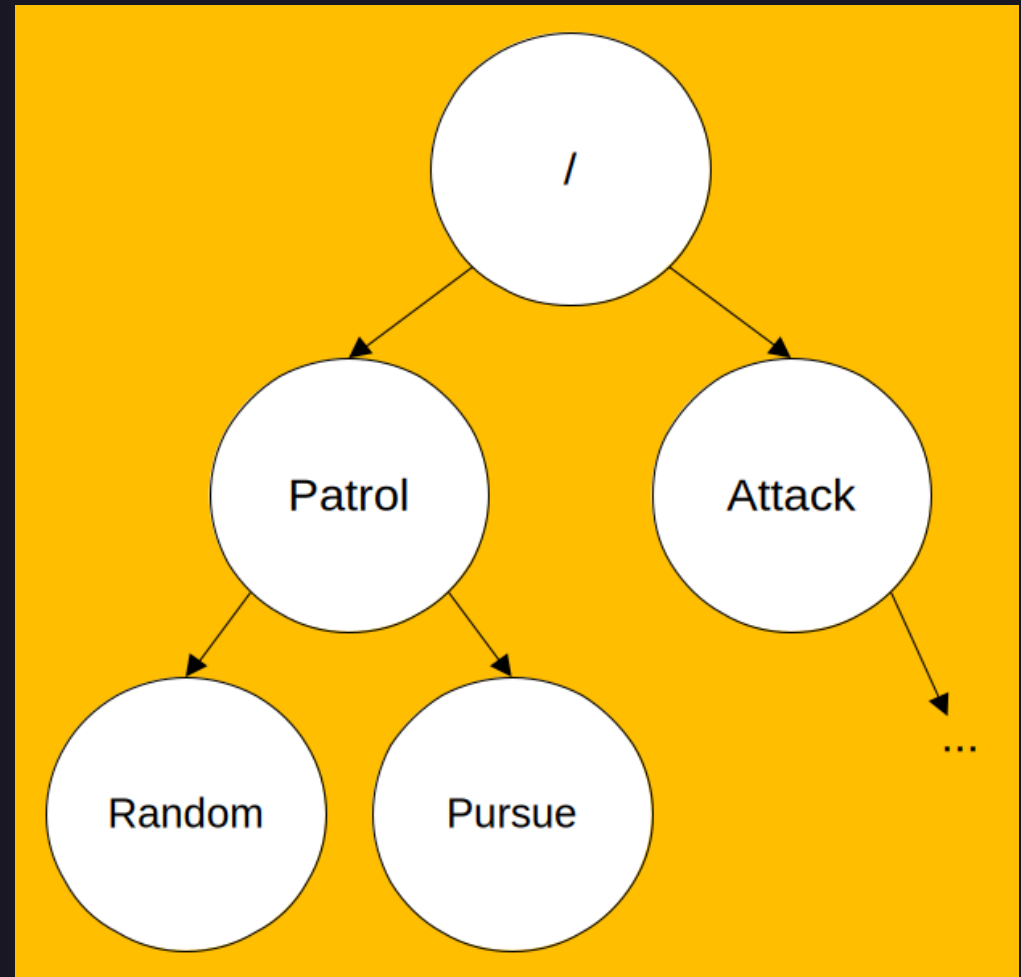
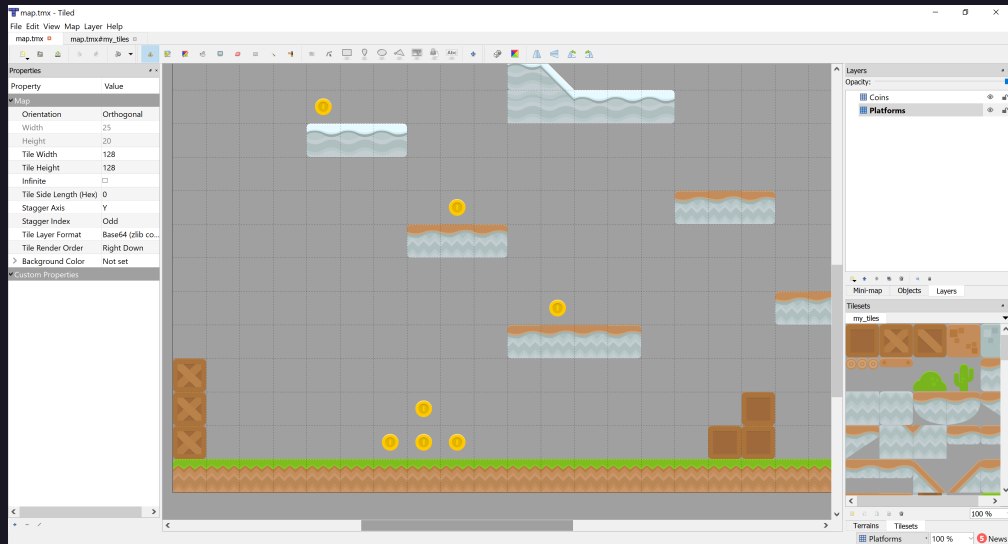
- Dungeon generation (functioning, optimization required)
- Enemy AI programming (hooks in place)
- GP co-evolution in progress

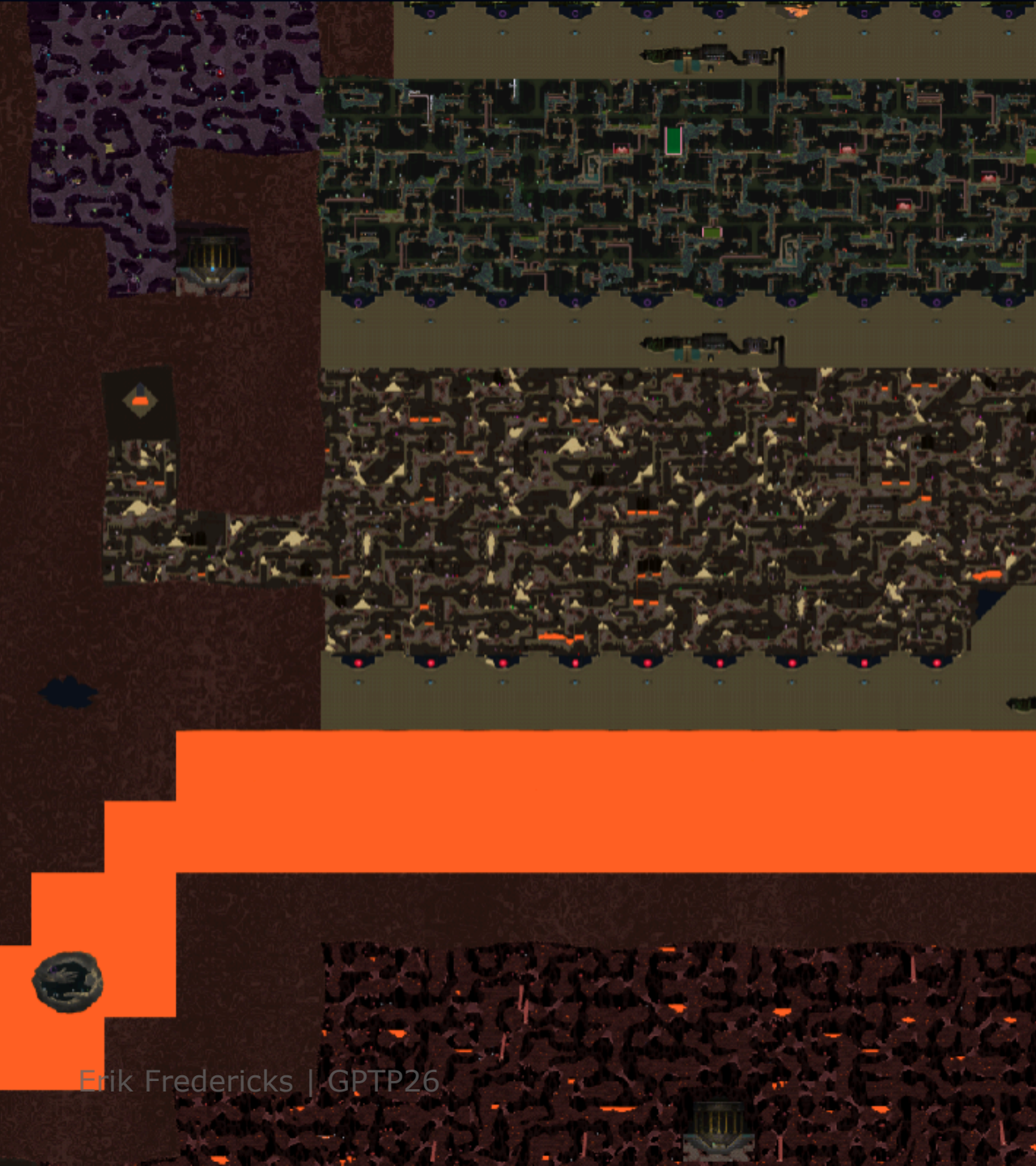


Why - making a game map/environment



Why - making a game map/environment





Erik Fredericks | GPTP26



x1?

Time: 00:32:32

Procedural content generation

What is PCG?

- Algorithmically-placing and generating content [hendrikx,shaker]

Why PCG?

- Save the developer/designer time and effort!
 - One would think...

Goal for GP?

- Optimize for variety and difficulty in game experience

Optimizing PCG

PCG is typically parameterized

Things we can optimize:

- **Ideal/diverse enemy behaviors**

- Behavior trees or programs

- E.g., [Move towards player, queue a bullet pattern, fire, wait 10 seconds, repeat]

- **Variety in environments**

- Obstacle/tile placement

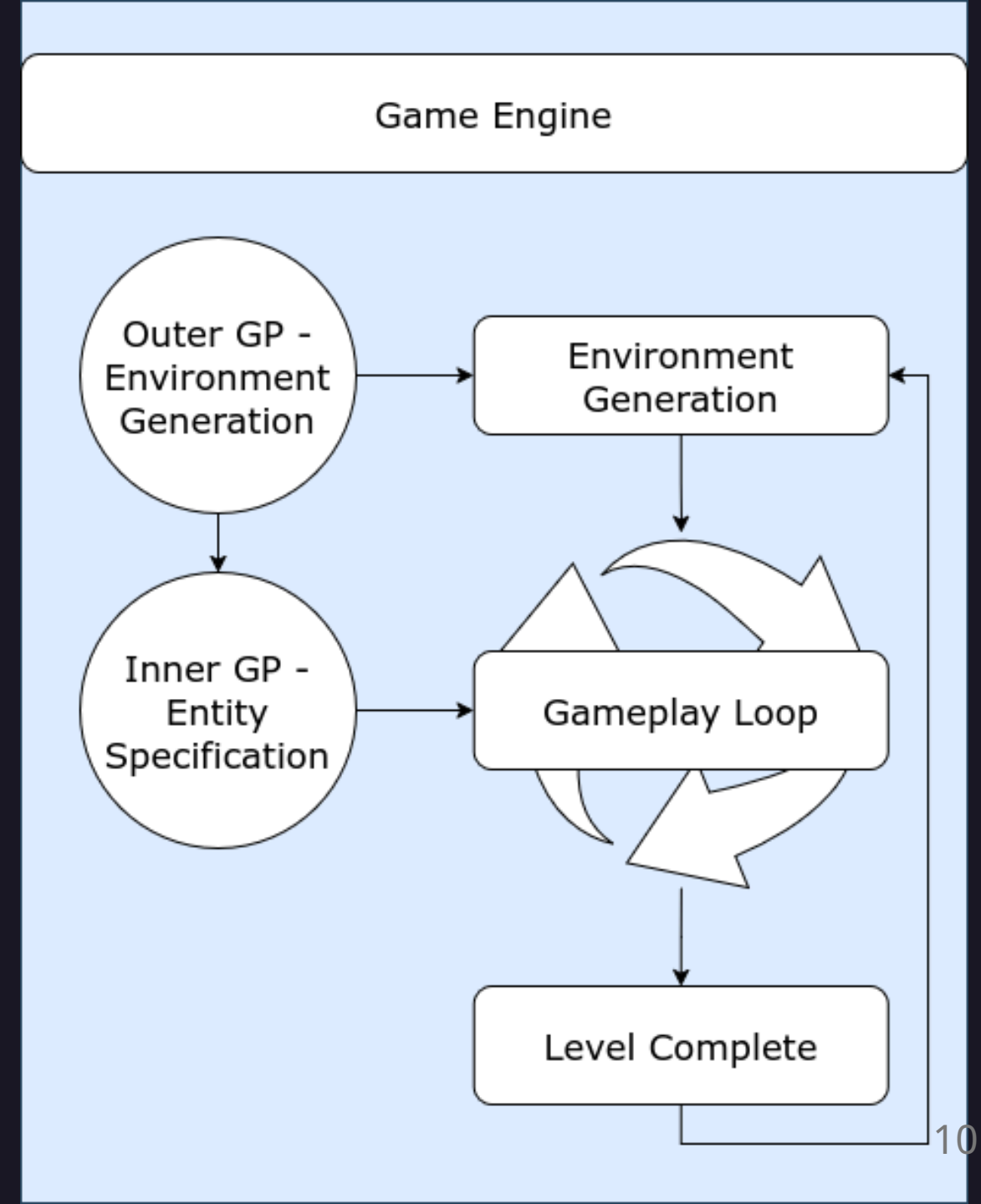
- E.g., #.t.\$e represents wall, space, gold, enemy
 - Represented as 2D/3D grid, graph, etc.

Prior Art

There has been a *lot* done with PCG and evolutionary computation

- **Cooke *et al.*** explored co-evolution for game generators []
- **Stanley *et al.***

Workflow



Genome

1. **Individual room layouts**

- 2D grid of characters to represent obstacles, painful obstacles, items, and enemies
- Room connections *currently* unoptimized

2. **Enemy programs** (tied to entities in room)

- List of actions to be executed in order
- Physics parameters (e.g., bullet speed, enemy acceleration, sensing radius)

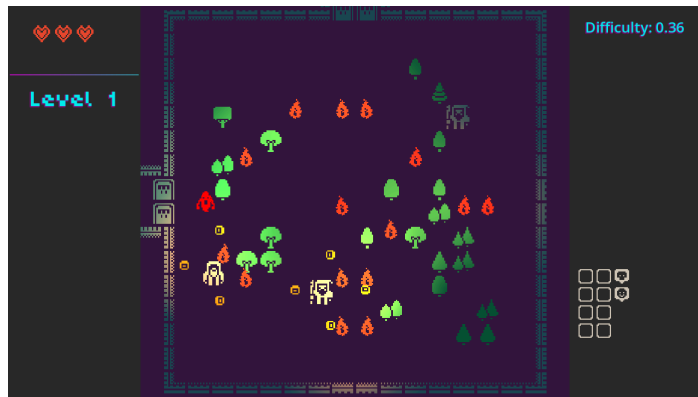
Fitness Functions

$$fitness = score_{layout} - penalty_{difficulty}$$

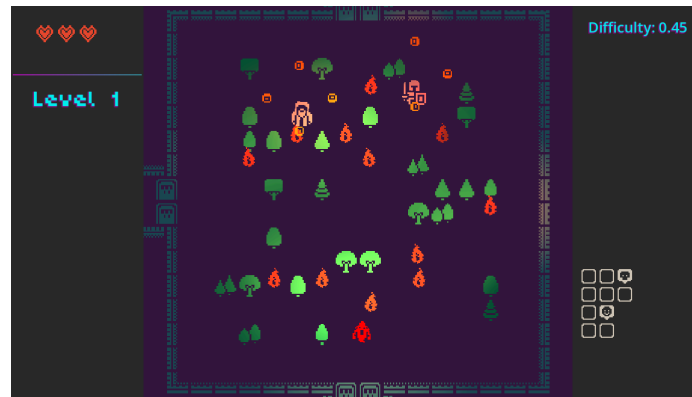
$$\begin{aligned} score_{layout} = & ratio_{obstacles} * w_1 \\ & + ratio_{pain\ obstacles} * w_2 \\ & + ratio_{open\ cells} * w_3 \\ & + number_{center\ obstacles} * w_4 \\ & + diversity_{enemy\ program} * w_5 \\ & + connectivity_{room} * w_6 \\ & + chokepoints_{room} * w_7 \\ & + clustering_{room} * w_8 \\ & + symmetry_{room} * w_9 \end{aligned}$$

$$penalty_{difficulty} = score_{difficulty} - target_{difficulty}$$

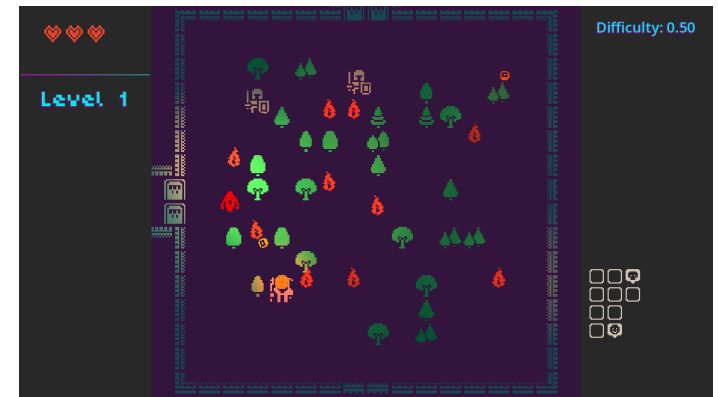
Sample Outputs (Room Generation)



Difficulty: 0.36



Difficulty: 0.45



Difficulty: 0.50

Sample Outputs (Enemy Program)

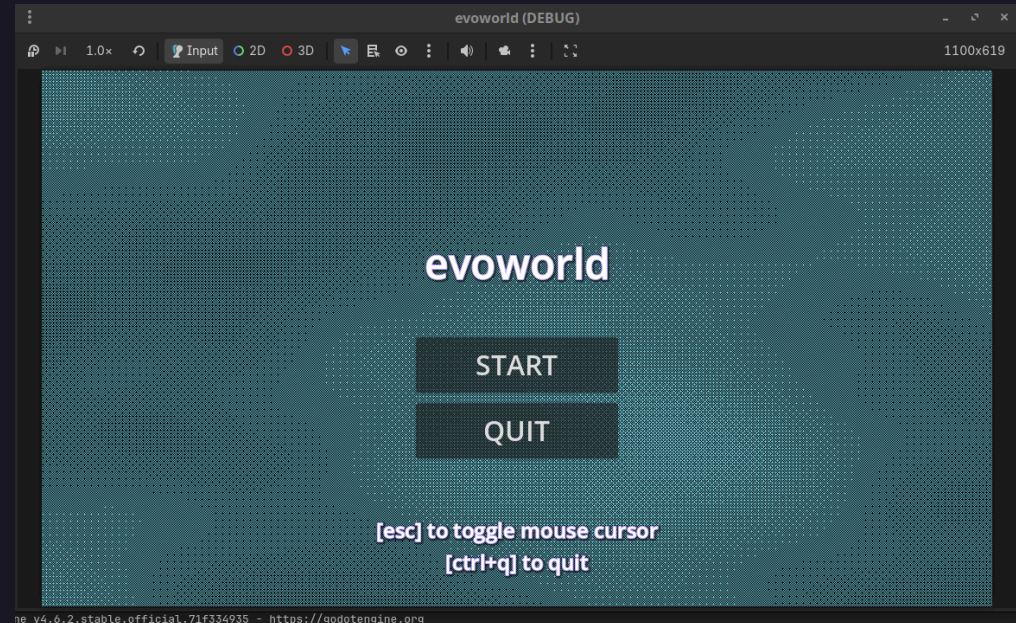
Enemy: WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, AIM, WAIT_20, FIRE_ONE

Enemy:

WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_AL
L, WAIT_40, WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, AIM, WAIT_20,
FIRE_ONE, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8
, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20
, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_4
0, WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, AIM, WAIT_20, FIRE_ONE, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40
, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_AL
L, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_
4, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_8, FIRE_ALL, WAIT_40, WAIT_20, RADIAL_4, FIRE_ALL, WAIT_40

Next Steps

- Update genome with feedback from gameplay in **current** room to impact **next** room
 - E.g., finished quick with no health lost? Next room is harder
- Additional fitness functions that reward room symmetry
- Log metrics for incorporating in future runs
 - Not just all sandboxed runs
- Full empirical evaluation



Ideal goal: Larger map worlds

Demo

The game is playable locally and in the browser

- Though, exiting just freezes the browser at the moment...
 - (works fine in local builds)
- Gameplay based on Zenva tutorial []

[**https://efredericks.github.io/gp-roguelite/**](https://efredericks.github.io/gp-roguelite/)

References

- [1] Cook, M., Colton, S., & Gow, J. (2016). The angelina videogame design system—part i. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(2), 192-203.
- [2] Cook, M., Colton, S., & Gow, J. (2016). The angelina videogame design system—part ii. *IEEE Transactions on Computational Intelligence and AI in Games*, 9(3), 254-266.
- [3] Togelius, J., Champandard, A. J., Lanzi, P. L., Mateas, M., Paiva, A., Preuss, M., & Stanley, K. O. (2013). Procedural content generation: Goals, challenges and actionable steps.
- [4] Stanley, K. O., Bryant, B. D., & Miikkulainen, R. (2005). Real-time neuroevolution in the NERO video game. *IEEE transactions on evolutionary computation*, 9(6), 653-668.
- [5] Pugh, J. K., Soros, L. B., Frota, R., Negy, K., & Stanley, K. O. (2017, September). Major evolutionary transitions in the voxelbuild virtual sandbox game. In *Artificial Life Conference Proceedings* (pp. 553-560).

Backup slides

PCG basic concept

Use math/algorithms to place content intelligently

- Dungeons, items, etc.

Noise functions:

- **Calculate a noise value based on inputs and configurations**
 - E.g., Perlin noise, Simplex noise,
- **Map that value to something useful**

For example - Perlin noise []

For example (in p5):

```
let n = noise(x * 0.01, y * 0.01);
```

- `n = [0.0, 0.5]` -- `(x, y) -> water`
- `n = (0.5, 0.6]` -- `(x, y) -> beach`

There are other noise functions! Worley noise, Simplex noise, etc.

Noise function mappings:

- Environment biome
- Map height
- Enemy spawns